

COMP-3150 CHAPTER 17: LECTURE NOTES (Ch. 17)

Chapter 17:

Indexing Structures for Files and Physical Database Design

The following questions on Indexing Structures for Files and Physical Database Design discussed in Chapter 17 of Comp 3150 text book, Chapters 17, Comp 3150 posted course slide notes, are in-class questions for students to ponder and answer as I teach.

- The answers to the questions are found also by reviewing the Comp 3150, posted power point slide notes for Chapters 17 and being in class.
- Students are advised to review Chapter 17 of course book and Comp 3150 posted slide notes before and after each class.
- I will also go over the Slide notes in class with examples and integrate them into the class lectures, which are also posted in the More course materials link on brightspace with any links to recorded live lectures.

1. How are database table records stored efficiently at the physical storage level on secondary storage such as disk in order to speed up data retrieval to answer queries?

2. Indexing is a mechanism used to reduce disk Input/Output time cost for answering queries.

i) What are indexes?

ii) What particular types of index structures are used in DBMS's?

- Example index structures for databases are B-Tree and B+-tree index structures.

3. What are B-tree and B+-tree index structures and what are the operations on these structures? Example operations are algorithms for loading data (inserting) onto a B-tree or B+-tree structure, for searching or retrieving data from a B-tree or B+-tree structure or deleting data from a B-tree or B+-tree structure.

- Focus of the Chapter 17 is on how to store database records efficiently on secondary storage such as disk in order to speed up data retrieval to answer queries.

- Locating data on disk is a major bottleneck in database applications.

- Thus, this Chapter looks at indexing as a mechanism to reduce disk input/output time cost for answering queries.

- Refer to Ch 17 slide notes on the course website or through brightspace.

Example 1: We want to answer the query: Get all employees with last name 'Zimmer'.

Solution 1: The primary data file for Employee table to search to answer query in Example 1 is stored as a disk file as shown in Fig 17.1 on slide 4 of Ch 17 slide notes.

- Assume there are about 1 million records in this file where the primary data file is a sequential file with attribute, Name as the primary key field.

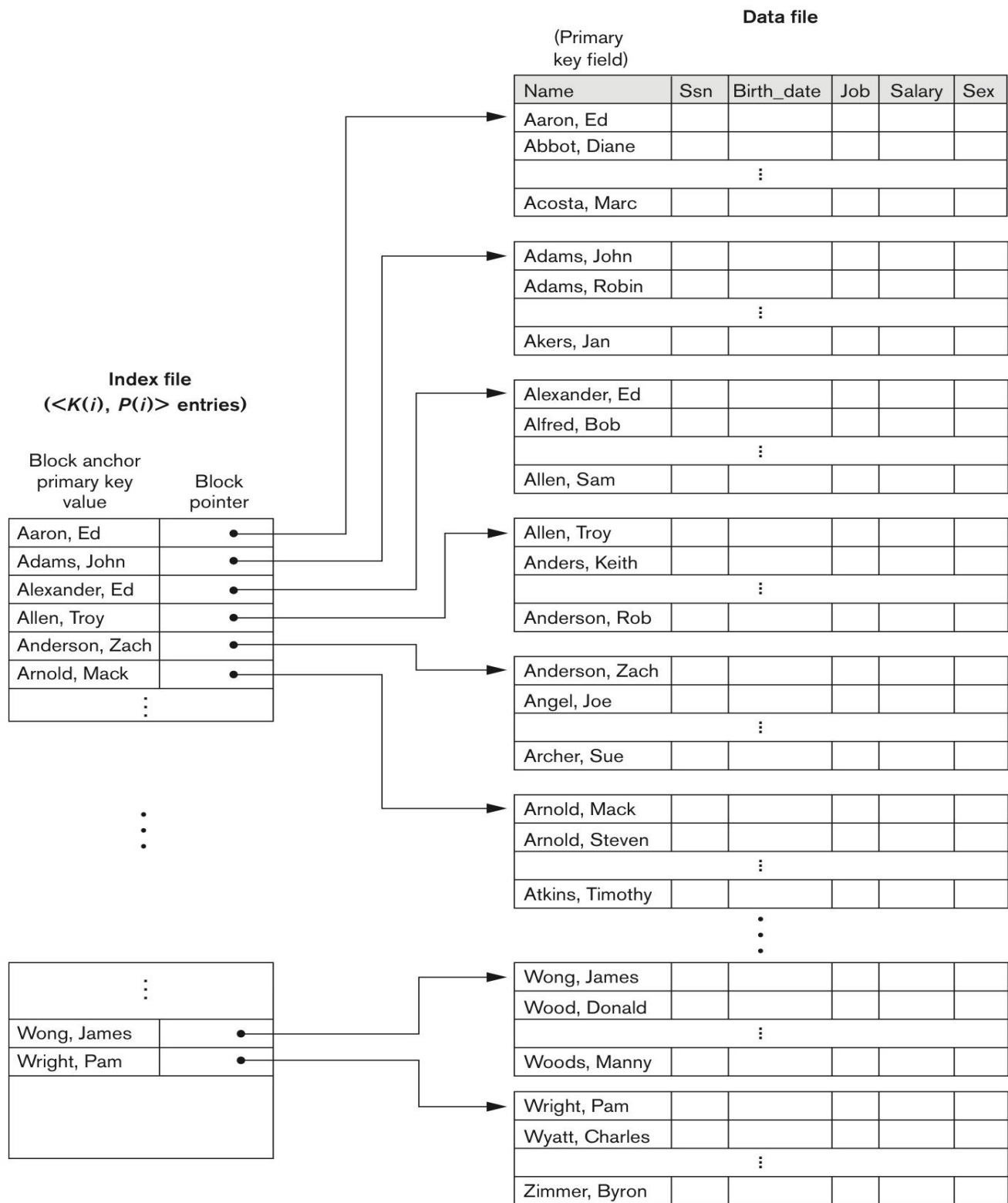
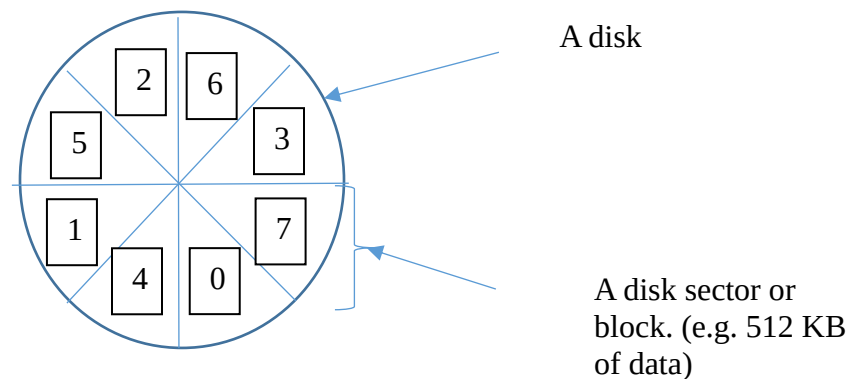


Figure 17.1 Primary index on the ordering key field of the file shown in Figure 16.7.

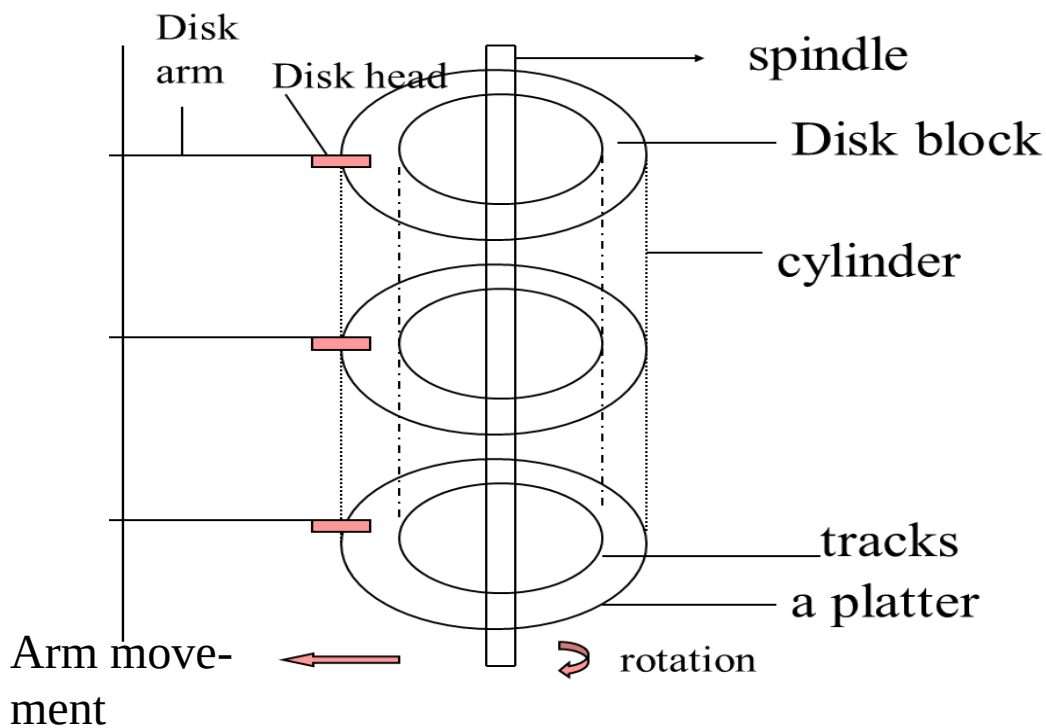
- To retrieve the last record with name 'Zimmer', the DBMS file structure would need to transfer nearly 1 million records from disk to memory in segments of disk block in order to look for the needed record.
- This operation of searching for the needed record stored in a sequentially ordered file on disk is very expensive and incurs huge disk I/O time cost.
- To fetch data on disk into memory, the disk address of block of data is given.
- The disk address of block data is usually, given in terms of the disk track numbers, sector number on the track, disk platter holding the track, etc.
- A disk track with 8 sector(blocks) can be formatted as shown below:



A Disk Track

- Disk input/output time (I/O time) consists of seek time + rotational delay time + data transfer time.

- The goal is to minimize this disk I/O time or time for accessing secondary storage using index structures.

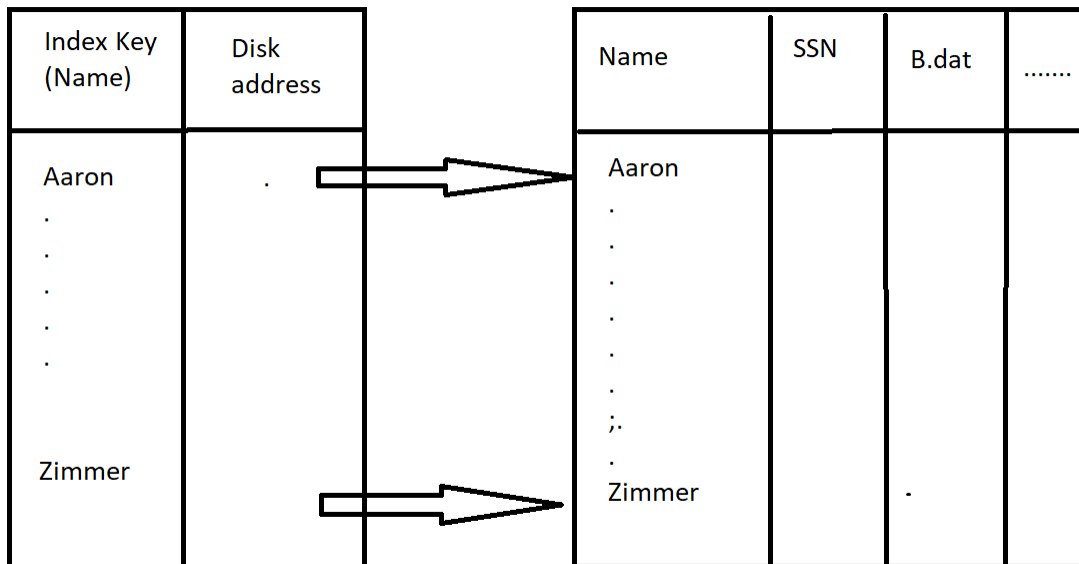


Structure of a Disk

2 (i) What is an index?

- Slide 3 of Ch 17 notes defines an index as an additional file on disk which provides secondary alternative access path to retrieve data (e.g., employee last name), that is on primary data file on disk through index key fields (e.g., last name).

- Any field or set of fields of the index file (e.g., Name) can be used as the index field.
- Going back to Example 1 using Figure 17.1, we can use an index file that has name of each employee with the disk address of where it is stored in the data file as shown below.



Check Fig 17.1

- Thus, this allows using an index file to answer a query more quickly on data in the primary employee data file using an index key attribute (Name) in the query.
- In that query's case, name is Zimmer in the index file, where the address of the disk block holding Zimmer in the primary data file is fetched so that this block is brought into memory directly instead of having to search sequentially about 1 million records before finding or not finding that record.

2 (ii) What particular types of index structures are used in DBMS's?

Types of indexes in general, are:

1. Ordered single level index files.
 2. Multi-level indexes (organized in tree structure that reduces the time for searching the index file to get the data block address).
 - These multi-level indexes are based on the binary search structure of cutting the search list into two during each iteration and continuing with only one branch and discarding the other.
 - With the multi-level indexes, the list can be partitioned into n parts (not just 2 parts as with the binary search).
 - B-trees and B⁺-trees are multi-level index structures commonly used in DBMS to implement dynamically changing indexes. They are balanced trees that handle frequent data insertions and deletions more gracefully with similar input/output (I/O) data access time since all leaves on a balanced tree are at the same tree depth level.
 3. Indexes can also be based on hashing or other search data structures
 - Here, a hash function is used to find the block address of the record being retrieved by the query.
 - An example hash function for a student simple record table based on the search index search key Stuid is:
 $\text{Stuid modulus } 100$
 - So, to find record of student with id 1003, DBMS looks in the disk block with hash address: $1003 \text{ modulus } 100$ or in block 3.
- Slide 7 of Ch 17 notes summarizes types of ordered indexes as:

1. Primary index: which are index files where the index key is the same as the primary key of an ordered data file of records (e.g., index key is Ssn on primary data file Employee).
 2. Clustering index: specified on ordering index key field (e.g., Dno) that is not primary key of the main file. The data file is called clustered file. Example is Fig 17.2 on slide 9.
 3. Secondary index can be specified on any non-ordering field of a file. Example is Fig 17.4.
- A data file can have at most one primary index or one clustering index but not both.
 - Slide 12 describes a multilevel tree index with f_0 (fan out) pointers and f_0 values as a tree with f_0 partitions of the search list.
 - An example multi-level index is Fig 17.6 (ISAM: indexed sequential access method) with $f_0 = 4$.
 - With the multi-level index of Fig. 17.6, each index page contains 4 index keys and 4 pointers that lead to the actual data page to be retrieved, on the primary data file with 13 pages (where each page holds 2 records).
 - For example, to get the record with the primary key 29, we start from the index root node and follow the highest index key of that page that is less than or equal to the key 29. This causes us to follow the key index of 2 at the top level, then, the key index 24 at the base level to find the page containing data records 24 and 29 from which we fetch record 29.

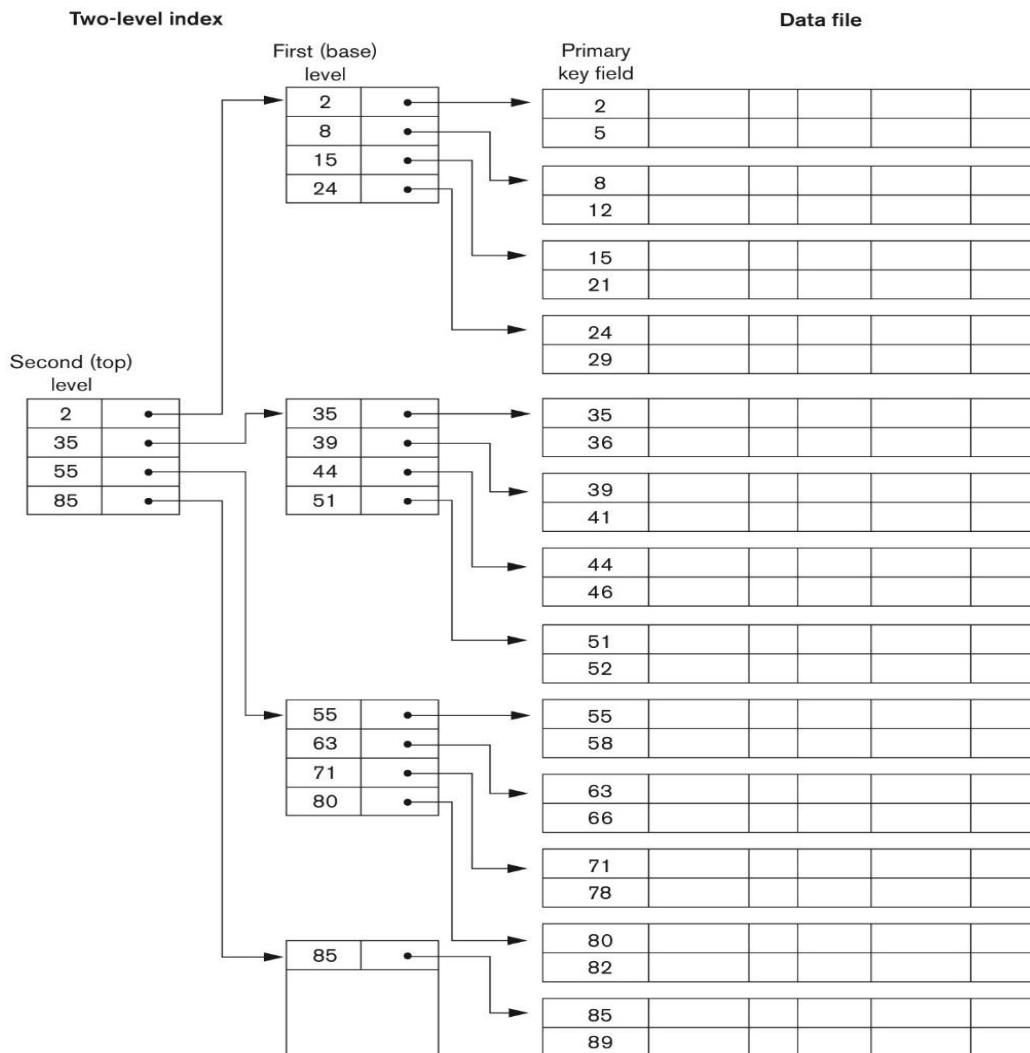


Figure 17.6 A two-level primary index resembling ISAM (indexed sequential access method) organization.

The multi-level index node format is like the following:

Structure of a Multilevel index Node with 4 key values and 4 pointers

K_1	P_1	K_2	P_2	K_3	P_3	K_4	P_4
-------	-------	-------	-------	-------	-------	-------	-------

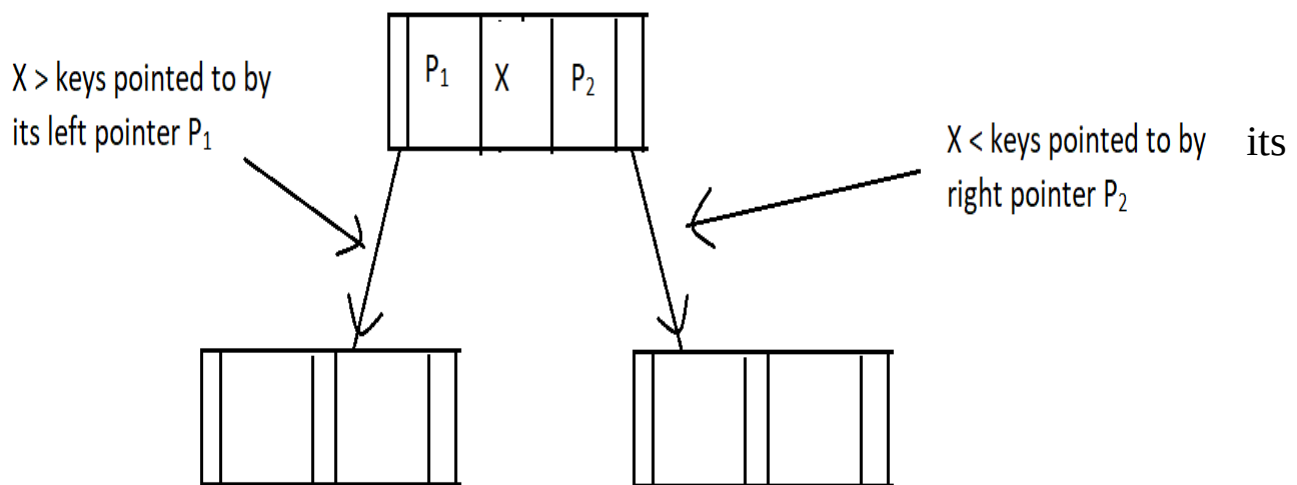
- That is, each K_i key value has its pointer P_i
- Multi-level indexes like ISAM are static structures that do not evolve with additions and deletion of the data in the database

- B-tree and B+-tree are dynamic balanced search trees with a slightly different structure as given below:

Structure of a B-tree index Node with 4 values and 5 pointers

P ₁	K ₁	P ₂	K ₂	P ₃	K ₃	P ₄	K ₄	P ₅
----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------

- Thus for the B-tree index with P pointers, each node has p-1 key values.
- All tree leaf nodes are at the same level.
- Slide 13 shows the properties of a node of the B-tree as:
 1. All key values have to be sorted as $K_1 < K_2 < \dots < K_{q-1}$
 2. For a search key X in a node, it has a left pointer and a right pointer



- Earlier, we discussed the importance of reducing disk I/O time using index structures.
- Two multi-level dynamic index structures commonly used in DBMS's are B-tree and B+-tree indexes.
- They are both balanced trees where all leaf nodes have the same depth level.

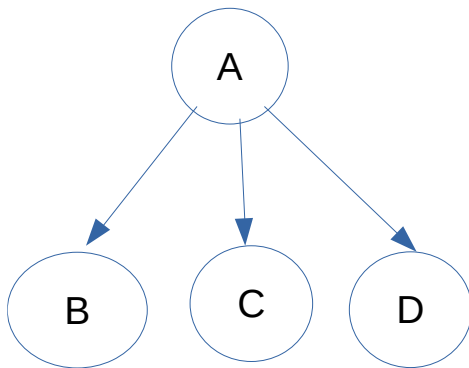


Fig 1: An example of a balanced tree

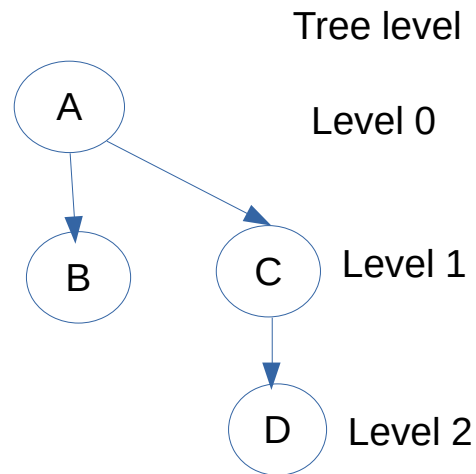
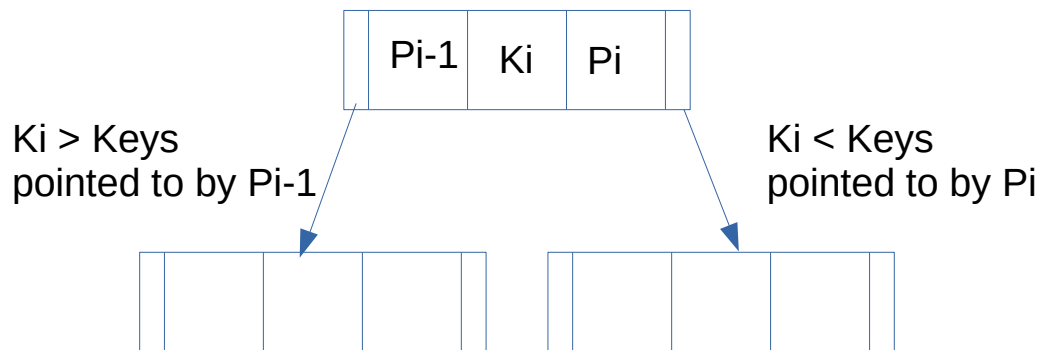


Fig 2: An example of an unbalanced tree

- Both B-tree and B+-tree require searching in the worst case $\log_{f_0}(b_i)$ disk block access to find the data or not, where b_i is the number of blocks (i.e, number of nodes or pages) in the index file; f_0 (fan out) is the number of tree pointers at each tree node (or page).
- Slide 16 of Ch 17 Slides presents the properties of a B-tree index of order P (that is, having P pointers and P-1 key values at each node).
- The 7 properties of B-tree are summarized as:
 1. Each tree node in B-tree (eg. Fig 17.10a on slide 17) is of the form: $\langle P_1, K_1, P_2, K_2, P_3, K_3, \dots, P_{q-1}, K_{q-1}, P_q \rangle$ for $q \leq P$, where each K_i consists of the pair of values $\langle K_i, P_{ri} \rangle$ meaning the key value and the data pointer or address of this K_i value on disk.

The P_i pointers in the property (1) of B-tree are the B-tree index pointers, while P_{ri} are data pointers.

2. Property 2 of the B-tree on slide 16 states that within each tree node, key values K_i are sorted in ascending order from left to right. $K_1 < K_2 < \dots < K_{q-1}$.
3. Property 3 states that a key value K_i is less than all key values in its right tree pointer but greater than values in its tree left pointer



4. Property 4 states that each node has at most P pointers
5. Property 5 states the minimum occupancy of a node with P pointers as $\text{ceil}(P/2)$ pointers. The root node can have at least 2 tree pointers
6. Property 6 states that if a node has $q \leq P$ (maximum allowed) pointers, it has $q-1$ key values.
7. Property 7 states that it is a balanced tree and all its leaf nodes are at the same tree level.

An example B-tree is Fig 17.10 on slide 17, which we shall build next.

Example 1

Build a B-tree of order $P=3$ for the data values 8, 5, 1, 7, 3, 12, 9, 6.

Solution 1

- Algorithm for inserting records into B-tree, deleting records from the tree or searching records in the tree are given in the book
- A summary of record insertion algorithm is given on slide 18 of Ch 17 notes. All of these algorithms focus on meeting the 7 properties of B-tree on slide 16. Insertion of a data value always starts looking for the leaf node to insert this data value from the Root node of the index tree.
- Inserting data into a full node causes a node split and only the middle value is pushed up to the parent node while the left split values stay in the left split node and the right split values stay in the right split node.

For example, to insert 1 into a full 2-record node that is already having the data records with ids 5, 8. Since this node is full, you imagine all three data values still in the ascending order as 1, 5, 8 before you have to split the node that should not have more than 2 items into two nodes at the leaf level, where the left split node holds 1 and the right split node holds 8; so that their middle element of 5 becomes their parent node in the tree.

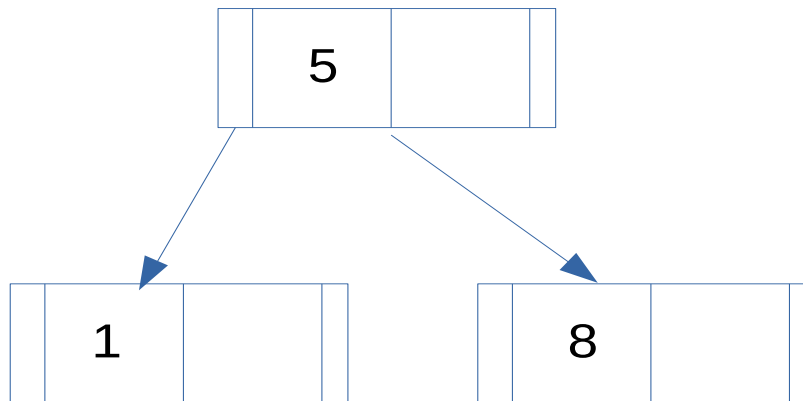
- Similarly, deletion of a key value from a node can cause a merge of nodes if the node becomes under-full by having less than 50% of P-1 key values (for example with P=3, the minimum occupancy of a node is 1 data element).
- We will only focus on insertion algorithm for now.
- Thus, to build a B-tree for (Order P=3) for index key values (such as student id) of **8, 5, 1, 7, 3, 12, 9, 6**; we proceed as follows:

Step 1: insert the first two values 8 and 5 into the root node as:

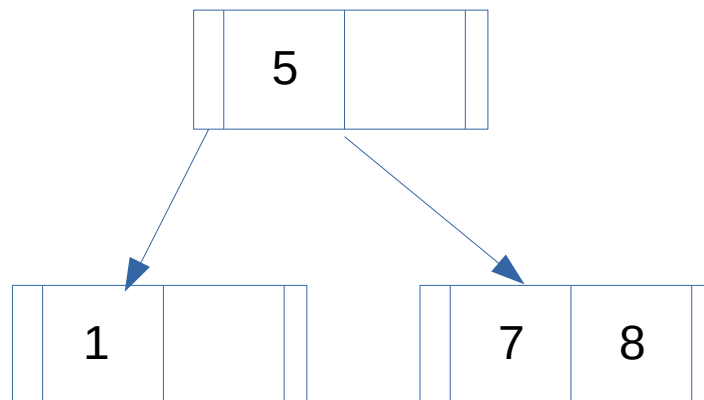
	5		8	
--	---	--	---	--

Step 2: insert 3rd key value 1 into the tree above in step 1. Following B-tree properties this tree node is full and has to be split into two nodes at a lower tree level than the root as **1, 5, 8**.

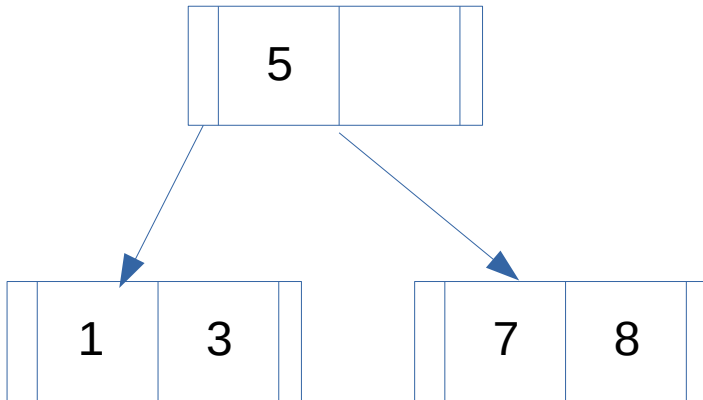
Note that 5 is the middle element and is pushed up as the parent node while 1 goes to the left split node and 8 goes to the right split node. Step 2 continues as node split to grow the tree as follows:



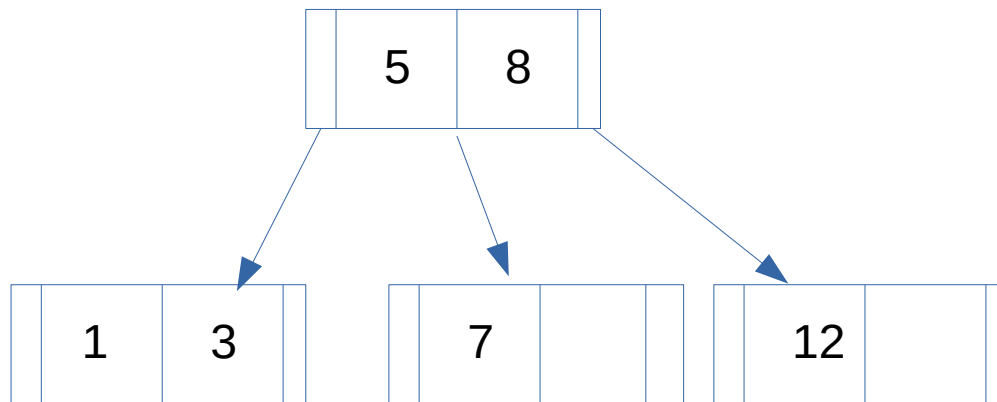
Step 3: insert the 4th record 7 into B-tree in step 2 as: look for the node it should go into starting the search from the root node, using the property that all data in right pointer of 5 are greater than 5. Since 7 is greater than 5 we follow the right pointer of 5. The new tree is:



Step 4: insert the 5th record 3 into the B-tree of Step 3 to get.

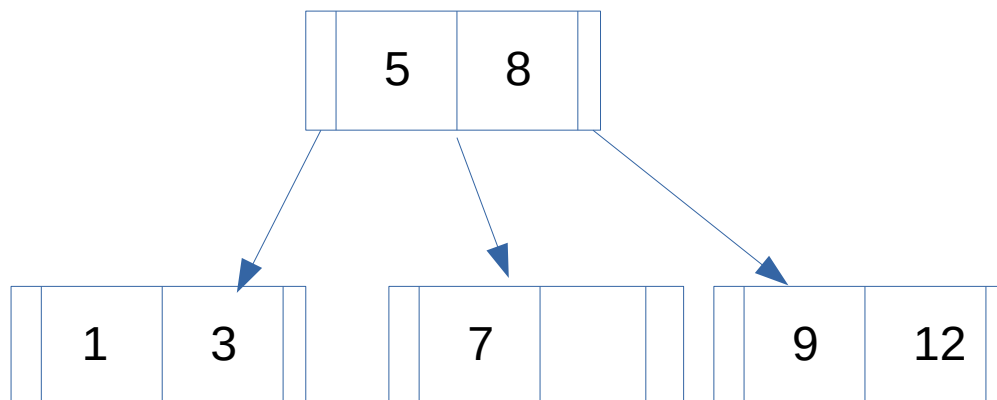


Step 5: insert the 6th element 12 into tree in Step 4. This should go to node 7, 8, 12 to cause a split as:

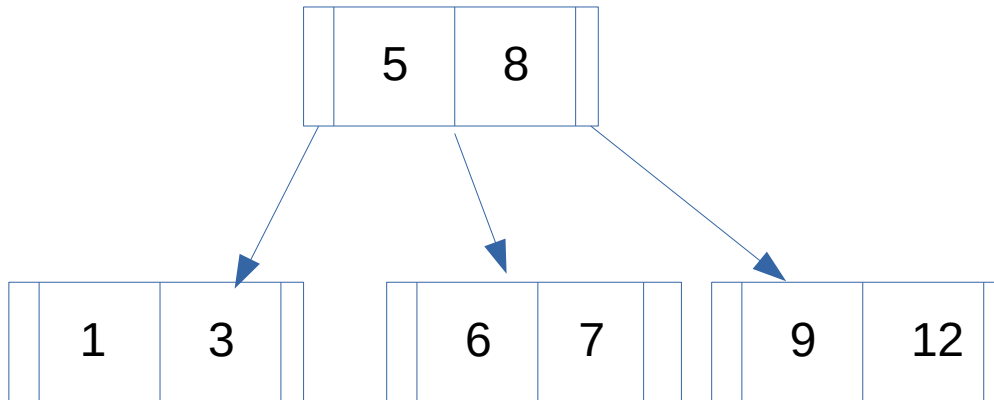


Note that the split causes the number of leaf nodes to increase to 3 from previous 2, and the middle element of the split node 8 moves up to be parent node of its two left split nodes.

Step 6; insert the 7th record 9 into tree of Step 5, to obtain:



Step 7: insert the 8th record 6 into tree of Step 6 to obtain:



This is the final B–tree as given in Fig 17.10 of book (and Slide 17).

The question on the final exam practice asks you to build onto this tree by inserting also records 2 and 10. Do this exercise.

Next, we build the B+-tree for the records

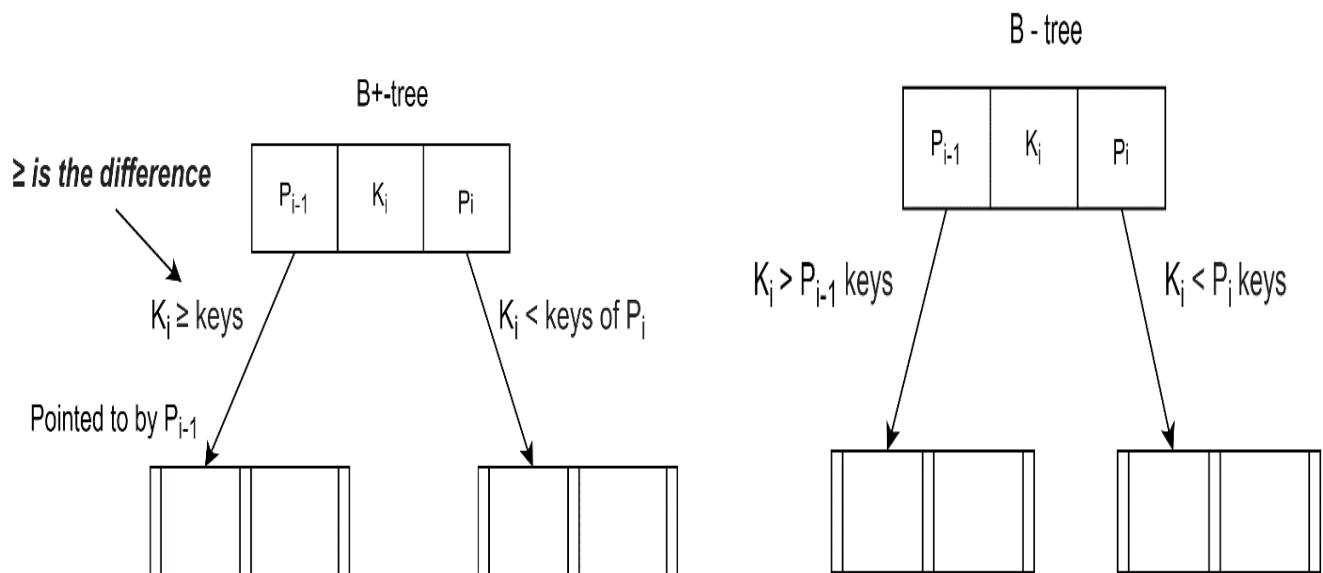
Ch 17: Indexing: DBMS Continues:

B+- tree Indexes

B+- tree

- Some of the differences between the B-tree and B+-tree include that all data values are found at the leaf nodes in B+- tree. These leaf nodes are also connected with pointers to answer range queries more easily.
- Thus, unlike the B-tree, B+-tree has both internal node and leaf node splitting algorithms. B-tree has only one splitting algorithm for both the internal and leaf nodes.
- With B+- tree, the node splitting algorithm for internal nodes is similar to that of the B-tree given on Slide 18 of ch 17 notes and discussed earlier but the node splitting algorithm for its leaf nodes is slightly different as given next.
- With B+-tree, the node splitting algorithm for leaf nodes is slightly different as it moves up the middle key value to the parent node but also keeps the middle element with the left split node at the leaf level since all data records have to appear at the leaf nodes. That is, all values from the first to the middle are in the left split node, while the rest are in the right split node.
- Slide 22 summarizes the differences between the B-tree and B+-tree.
- Another difference between the B-tree and B+-tree is in the structure of the internal node where for B+-tree, a key value K_i is less than all key

values in its right tree pointer but greater than or equal to the key values in its left pointer as shown in the figures below.



- Now Example 1 on B+-tree:

Build a B+-tree of order $p=3$ (three pointers per node) for the search key values.

8, 5, 1, 7, 3, 12, 9, 6

- Solution 1:

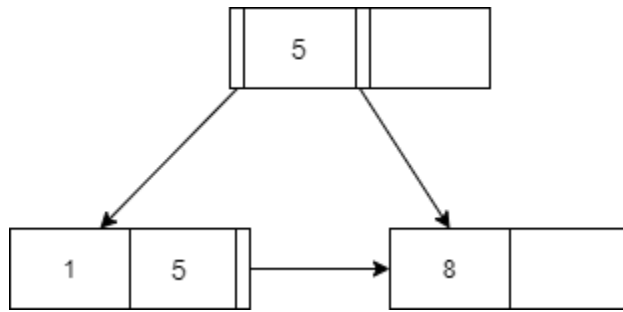
Step 1: Start inserting each key value into B+-tree nodes starting from the first two values 8 and 5 to obtain the tree:

5	8
---	---

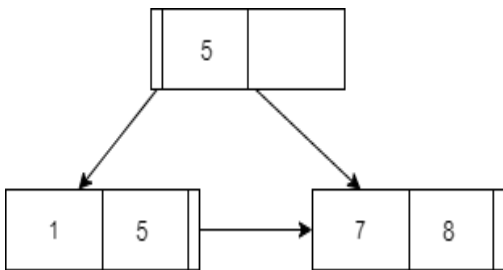
 → This is the root and a leaf node which has no tree pointer between key values on the same node. It has tree pointer to the next leaf node for answering range queries. All data values have data pointers for retrieving them from disk.

- Step 2: Insert the third key value of 1 into the B+-tree in Step 1.

Since this leaf node is full as 1, 5, 8, we must split it into two leaf nodes where the mid element goes up to the parent and the left split node has 1 and 5, while the right split node has 8.



- Step 3: Insert the key 7 into the B+-tree of step 2 as:

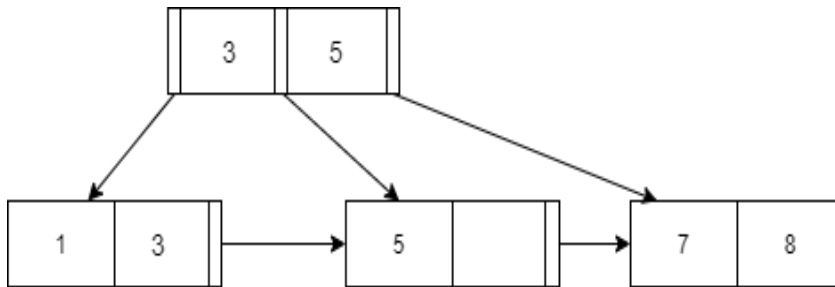


Using the B+-tree Search algorithm, start from the root key value 5. Since 7 is greater than 5, follow the right pointer of 5 to find the node to insert 7 in sorted order.

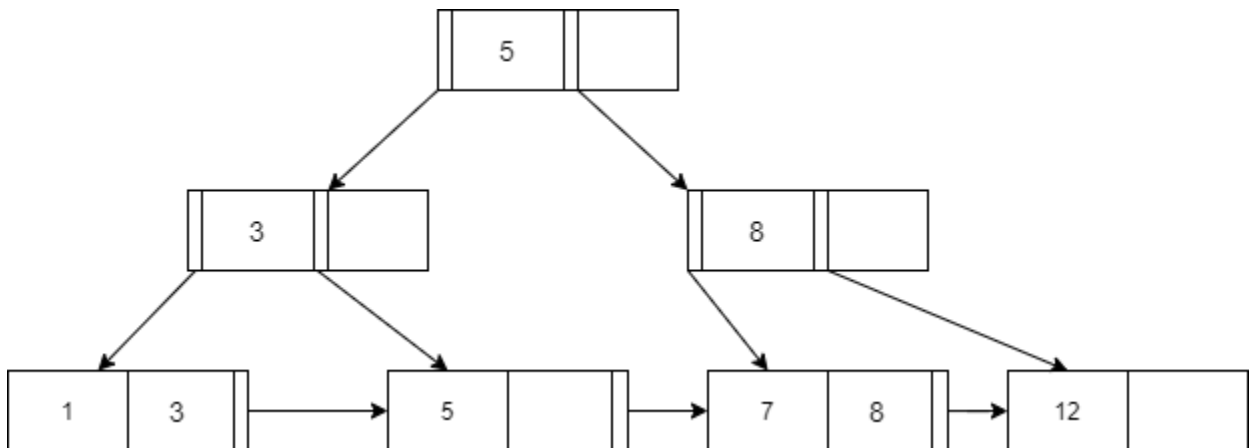
- Step 4: Insert the 5th element 3 into the node 1, 3, 5, to cause a split at the leaf level.

Note that all insertions start at the root node to look for the node to insert 3. Since 3 is < the key value 5 at the root, we follow the left pointer of 5 at root, to find the node to insert 3 at the leaf. Because it is

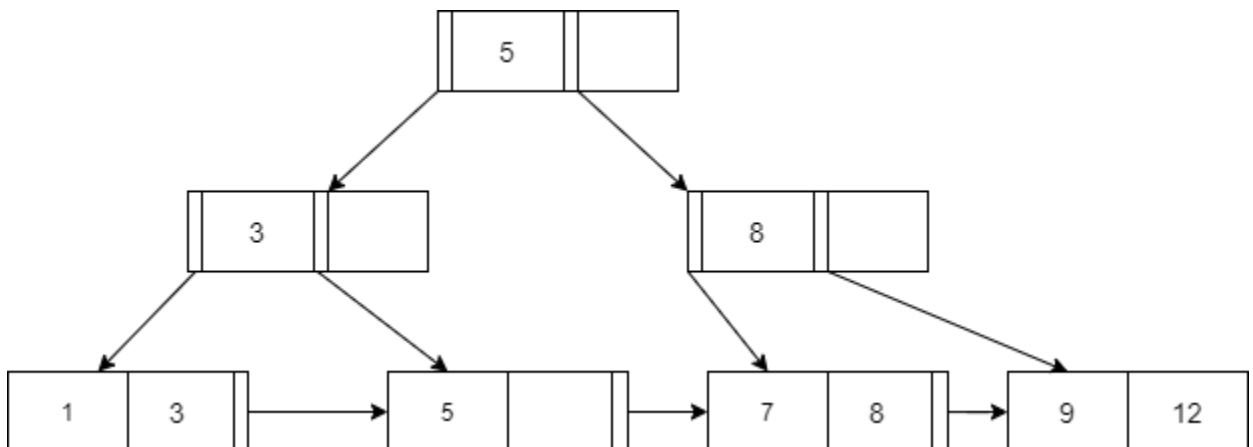
full, we split it as:



- Step 5: Insert 12 into leaf node 7, 8, 12 to cause a split at leaf. Also, key value moves up to internal node 3, 5, 8.

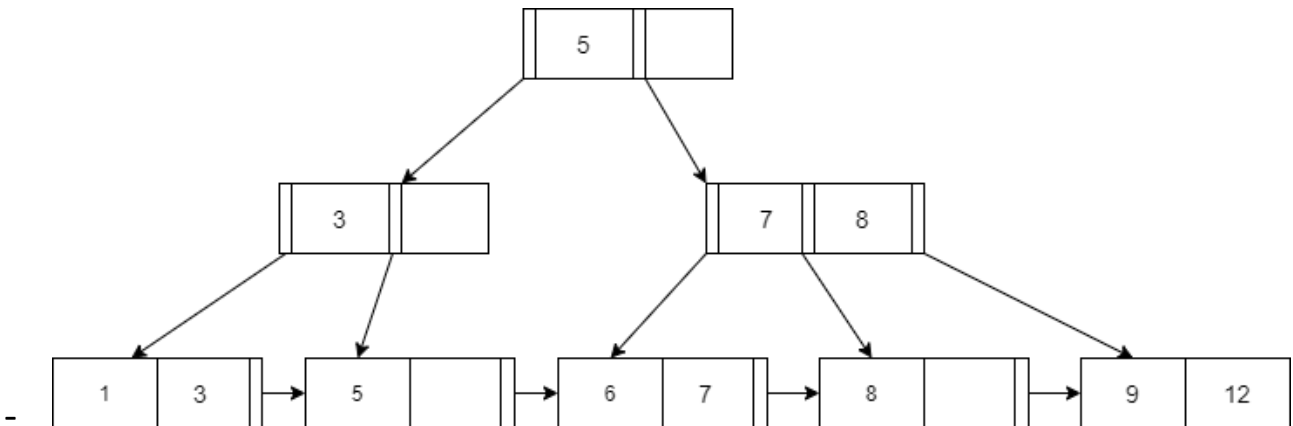


- Step 6: Insert the 7th element 9 into the B+-tree of step 5 to get



-

Step 7: Insert the 8th record 6 into the B+-tree of step 6. Following the right pointer of the root key 5, we get to internal node key 7 where we must follow its left pointer to get to node 6, 7, 8 that must split into two at leaf level as shown below.



- This is the final B+-tree for 8, 5, 1, 7, 3, 12, 9, 6 as Fig 17.12 in the book.

As exercise,

- Continue to insert 2 and 10 for final exam practice. END

EXTRA LEARNING

Can we then try to review more work on building the B-tree and the B+-tree as given in question 4 of the posted practice final examination.

Question 4 : (total marks = 15 for 7.5 + 7.5)

(a). Given the following unique id for some database records of a relation, construct the B-tree index structure of order $p=3$ (or with maximum of 3 pointers) for speeding up retrieval of these records from their primary data file stored on disk. The data to build index file for are:

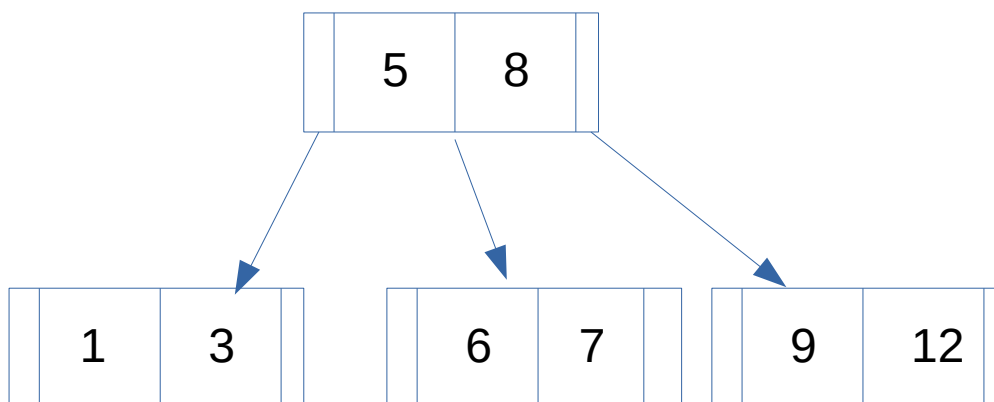
8, 5, 1, 7, 3, 12, 9, 6, 2, 10

Show the steps through arriving at your final tree before giving your final tree. (7.5 marks)

Solution:

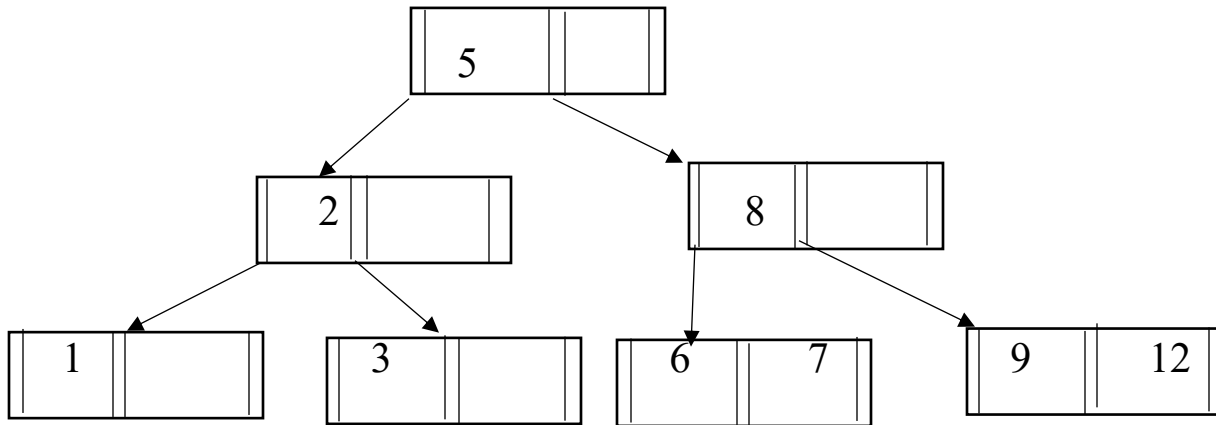
Previously, in class, we had constructed up to 6 to have:

8, 5, 1, 7, 3, 12, 9, 6 as the following tree to build on by inserting, 2, 10 into.

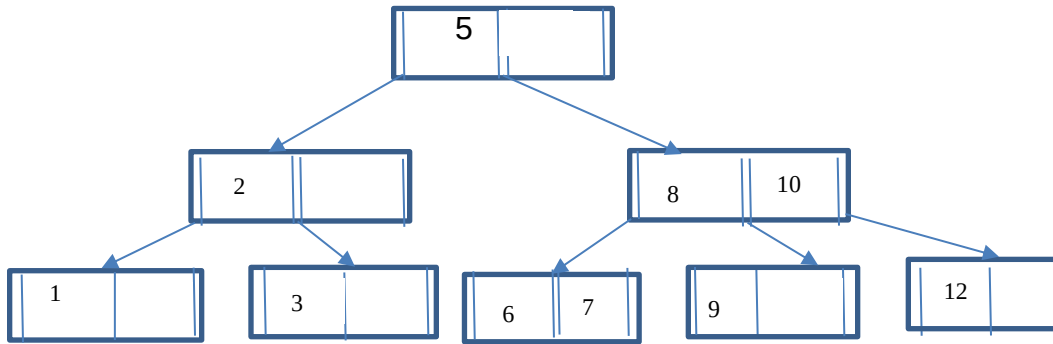


Now insert 2 and 10 into this B-Tree

Insert 2: Follow the root key index 5 left pointer to insert 2 into the leaf node 1 2 3 to cause a split of that node into nodes 1 and 3 with the parent 2 to be inserted into the root node 2 5 8 to cause a split at that level into two internal nodes of 2 and 8 with their parent as 5 (new root)



Insert 10: Insert into the node 9 10 12 to cause a split where 10 becomes the parent of split nodes 9 and 12. To obtain the final B-tree for 8, 5, 1, 7, 3, 12, 9, 6, 2, 10 as:

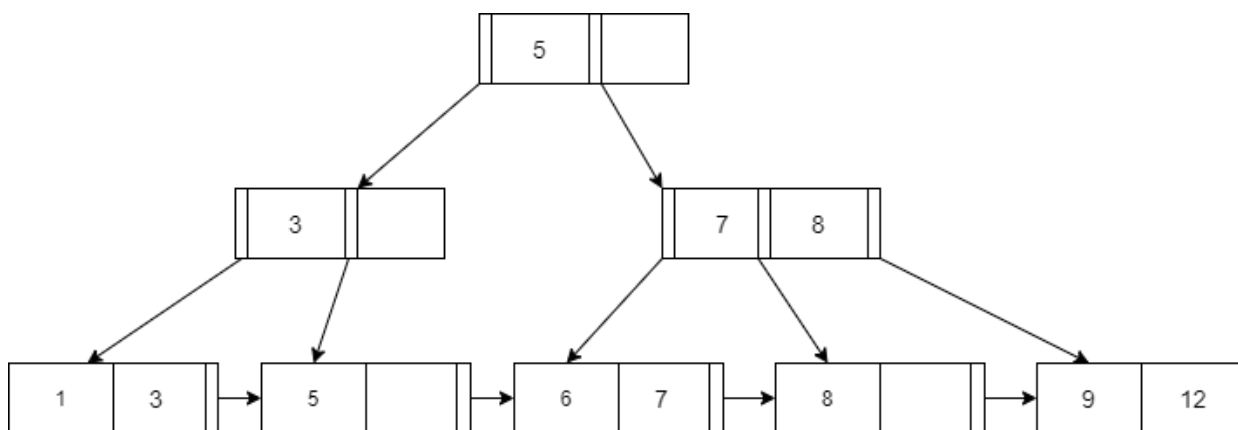


This is the final B-tree after inserting 8, 5, 1, 7, 3, 12, 9, 6, 2, 10

(b). Using the same data set of 8, 5, 1, 7, 3, 12, 9, 6, 2, 10, construct a B⁺-tree index structure of order $p=3$ (or with 3 [pointers for internal nodes and 2 data entries for leaf nodes). Show the steps through arriving at your final tree before giving your final tree. (7.5 marks).

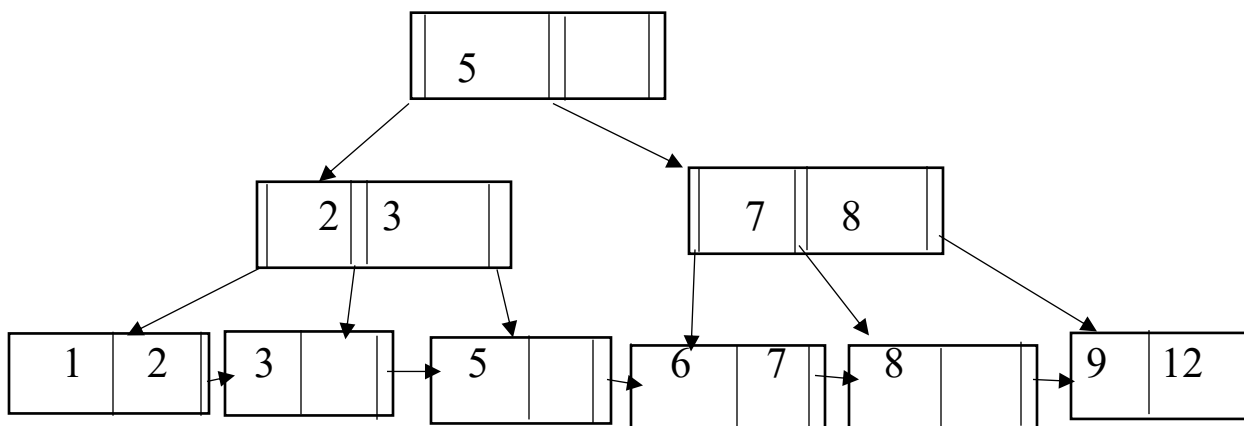
Solution:

Previously, in class, we had built a B⁺-tree for the records: 8, 5, 1, 7, 3, 12, 9, 6 as follows:

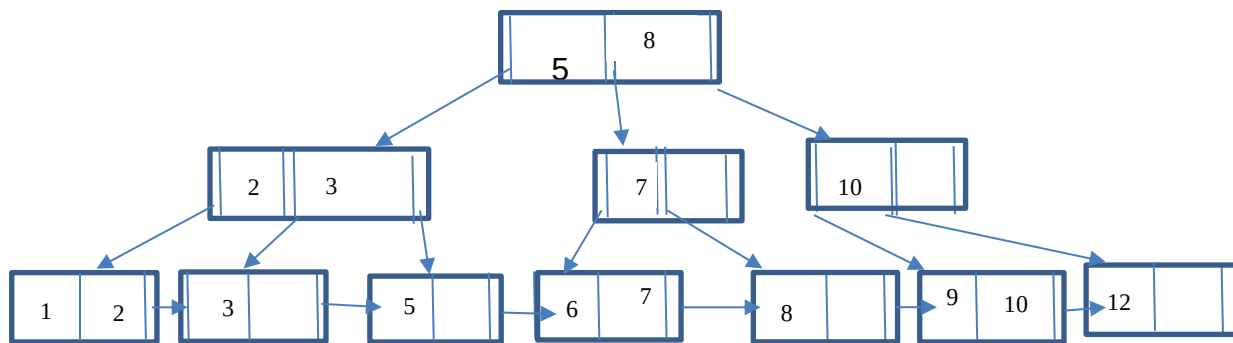


The task now is to insert the two numbers 2 and 10 to the above tree.

Insert 2: From root, we follow the left pointer of 5 to find the node to insert 2, which is node 1, 2 3 that should be split into two nodes at the leaf (where the middle element is also kept with the left split node before being pushed up as the parent node) 1, 2 and 3 with 2 as their parent.



Insert 10: Insert 10 into the node 9, 10, 12 to cause a leaf split as nodes 9, 10 and 12. Then, push up to the parent node 10 to also cause an internal node split of node 7, 8, 10 to obtain nodes 7 and 10, where index value 8 moves up to the root node to become the parents of nodes 7 and 10. The final B+-tree after insertion of 2 and 10 becomes:



This is the final B+-tree after inserting 8, 5, 1, 7, 3, 12, 9, 6, 2, 10

**

Extra: About Deletions on B-Tree and B+-Tree index trees (Deletion from trees not part of exam)

- Recall that for B-tree index structures, as stated on slide 19 of Chapter 17 slide notes, each B-Tree node can have at most p tree pointers, $p-1$ data pointers and $p-1$ search key values (see Fig 17.10), Fig 17.12 and Fig. 17.13. This is slightly different for each B+-tree node where the internal nodes have only p tree pointers but no data pointers since all data also appear at the leaf nodes; while for the B+-tree leaf nodes, there are no p tree pointers but a pointer from one node to the next node and $p-1$ data pointers, as shown in Figure 17.12 on Slide 20.
- Just as insertion of data into tree nodes can cause a split of the node if the node is full (has more than $p-1$ key values), deletion of data from the tree can also cause a merge of nodes if the node is under full (has less than 50% of $(p-1)$ key values). Deletion of data from nodes can also be handled through data re-distribution of merged nodes at the same tree level that still meets the requirements of the index tree, if possible.
- Fig. 17.13 on page 21 of Ch. 17 slide shows example deletions of the sequence of data values 5, 12, 9 from the B+-tree of 1, 5, 6, 7, 8, 9, 12.

-

Extra for Question 3: (c). Retrieve the average quantity of items ordered by each customer

SQL:

```
SELECT C.Cust#, C.Cname, avg(R.Qty)
FROM CUSTOMER C, ORDER O, ORDER_ITEM R
WHERE C.Cust#=O.Cust# AND O.Order# = R.Order#
Group by C.Cust#, C.Cname;
```

Relation Algebra:

$$\text{Cust\#,Cname} \bowtie \text{avg Qty} (\text{CUSTOMER} \bowtie_{\text{cust.cust\#}=order.cust\#} (\text{ORDER}) \\ \bowtie_{\text{Order.Order\#}=order_item.order\#} (\text{ORDER_ITEM}))$$

Relational Calculus:

$$\{c.\text{Cust\#}, c.\text{cname}, \text{avg}(R.\text{Qty}) \mid \text{customer}(c) \text{ and } \text{order_item}(R) \\ \text{and } \exists(O) (\text{order}(O) \text{ and } c.\text{cust\#} = O.\text{cust\#} \text{ and } O.\text{order\#} = R.\text{order\#})\}$$

Query Result:

Cust#	Cname	avg(Qty)
1	Peter	1
2	Mary	2